ChaCha20 アルゴリズムにおける ARX 構造検出手法

長島 更紗[1,a)]　小林 良太郎[1,b)]　加藤 雅彦[2,c)]

概要：ランサムウェアとは，データを暗号化し，復号と引き換えに身代金を要求するマルウェアの一種である．近年では多重脅迫やノーウェアランサム，Ransomware as a Service (RaaS) など，その脅威は高度化，巧妙化している．本研究では，暗号化型ランサムウェアで使用されている暗号化アルゴリズムの 1 つである ChaCha20 に着目し，そのアルゴリズムの核となる ARX 構造（Add, Rotate, Xor）の検出手法を提案する．ChaCha20 プログラムに対して，ARM 環境の逆アセンブル結果と QEMU トレースを解析し，ARX 構造を検出することを目的とする．さらに検出プログラムによる ARX 構造の検出数を用いて提案手法の評価を行う．

キーワード：ランサムウェア, ChaCha20, ARX 構造

ARX Structure Detection Method in the ChaCha20 Algorithm

Sarasa Nagashima[1,a)]　Ryotaro Kobayashi[1,b)]　Masahiko Kato[2,c)]

Abstract: Ransomware is a type of malware that encrypts data and demands a ransom in exchange for decryption. In recent years, the threat posed by ransomware has become increasingly advanced and sophisticated, as exemplified by multiple-extortion tactics, no-ware ransomware, and Ransomware as a Service (RaaS). This study focuses on ChaCha20, one of the encryption algorithms used by crypto-ransomware, and proposes a method for detecting the ARX structure, which forms the core of the algorithm and consists of addition, rotation, and exclusive-OR operations. Specifically, we aim to detect the ARX structure in a ChaCha20 program by analyzing its disassembly output and QEMU execution traces obtained in an ARM environment. Furthermore, we evaluate the proposed method using the number of ARX structures detected by the developed detection program.

Keywords: Ransomware, ChaCha20, ARX Structure

1. はじめに

近年，ランサムウェアの被害が大きな注目を集めている．日本国内でも中小企業をはじめ，多くの企業でランサムウェア被害が発生している．また飲料メーカーや通販企業，出版社，自動車メーカーなど，大手企業の被害でさらに注目を集めている．ランサムウェアとは，パソコンなどに保存されているデータを暗号化して使用不能にし，元に戻すことを代償に金銭を要求するマルウェアのことである．近年では，従来の暗号化解除についての脅迫に加え，暗号化前に窃取した重要データの公開について脅迫する二重脅迫も出現している．さらに顧客や取引先に圧をかける三重脅迫や，ランサムウェアを仕掛けた会社に対し追加で Distributed Denial of Service (DDoS) 攻撃を仕掛け，企業のサービスを停止させる四重脅迫など，多重脅迫の手口も出現している．

ランサムウェアには暗号化対象や被害によっていくつかの種類があるが，大きく分けて画面ロック型と暗号化型の 2 種類に分けられる．画面ロック型は，パソコンやスマー

[1] 工学院大学
Kogakuin University, Shinjuku, Tokyo 163-8677, Japan
[2] 順天堂大学
Juntendo University, Bunkyou, Tokyo 113-8421, Japan
[a)] j123227@ns.kogakuin.ac.jp
[b)] ryo.kobayashi@cc.kogakuin.ac.jp
[c)] m.kato.ug@juntendo.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

トフォンなどの端末の画面をロックして一切の操作を不能にし，その解除コードと引き換えに身代金を要求する手口である．暗号化型はデータやファイルを暗号化し，データの復号と引き換えに身代金を要求する手口である．また，近年はデータを暗号化せずに窃取し，対価を要求するノーウェアランサムも増加傾向にある．これらの被害は，感染後に身代金の支払いを行っても，正常なシステムやデータの復旧は保証されない．また，調査費用や復旧費用は高額となり，復旧までの時間も長期化していて被害規模は甚大である．

今回は特にランサムウェア被害の大半を占める暗号化型に着目する．暗号化型ランサムウェアで使用されている暗号化するアルゴリズムの代表例としては AES, RSA, ChaCha20 があげられる．AES, ChaCha20 はどちらも共通鍵暗号方式である．AES はハードウェア処理に適していて，AES-NI などの AES 専用の命令セットなどのハードウェア支援がある環境では極めて高速に動作するが，スマートフォンや Internet of Things (IoT) 環境など，そうではない環境では暗号化が遅い．一方 ChaCha20 はソフトウェア処理に長けていて，ハードウェア支援の有無にかかわらず高速である．そのため，近年の IoT 増加や，Ransomware as a Service (RaaS) などの普及に伴い，ファイル暗号化に ChaCha20 を使用する事例が確認されている．Alzahrani らは Conti ランサムウェアの流出コードを解析し，ファイル暗号化に ChaCha20 を使用し，生成した暗号鍵を RSA で暗号化する方式が実装されていることを示している．[1] 本研究では ChaCha20 に着目し，そのアルゴリズムの核となる ARX 構造の検出手法を提案する．IoT 環境を想定し，ARM 向けにコンパイルされた実行ファイルの逆アセンブル結果と ARM 環境をエミュレートした QEMU での実行トレースを用いて命令列に着目した解析手法である．

本論文の構成は以下の通りである．第 2 章で ARX 構造について，第 3 章で関連研究について述べ，第 4 章で提案手法について説明する．第 5 章では提案手法の検証を行い，第 6 章で検証結果について述べる．また，第 7 章で考察，第 8 章でまとめおよび今後の課題について述べる．

2. ARX 構造

ChaCha20 アルゴリズムは定数，256 ビットの秘密鍵であるキー，96 ビットの同一の鍵に対して 1 回のみ使用できるナンス，32 ビットのカウンタを入力として，疑似乱数である鍵ストリームを生成する [2]．ChaCha20 は暗号化と復号に同じ共通鍵を用いる．

Algorithm 1 ChaCha20 algorithm

```
1: QuarterRound(0, 4, 8, 12)
2: QuarterRound(1, 5, 9, 13)
3: QuarterRound(2, 6, 10, 14)
4: QuarterRound(3, 7, 11, 15)
5:
6: QuarterRound(0, 5, 10, 15)
7: QuarterRound(1, 6, 11, 12)
8: QuarterRound(2, 7, 8, 13)
9: QuarterRound(3, 4, 9, 14)
```

ChaCha20 は列成分と対角成分に対して交互に 20 ラウンド実行する．各ラウンドは Quarter Round 4 回で構成されていて，引数として 4 つの値を受け取る Quarter Round を 4 回繰り返すことで 16 個の数字をシャッフルできる．上記のアルゴリズム 1 は関数呼び出し部分を表しているが，1 行目から 4 行目が列成分のラウンド，6 行目から 9 行目が対角成分のラウンドを表している．

また，この QuarterRound 構造は加算 (`add`)，排他的論理和 (`eor`)，左回転操作 (`rol`) からなる ARX 構造で構成される．以下のアルゴリズムは C 言語を模倣した ARX 構造の疑似アルゴリズムである．1 行目の `a += b` は a に a+b を代入するという意味である．続いて，`d ^= a` は d に a と d の排他的論理和を代入し，`d <<<= 16` は，d を 16 ビットの左シフト回転する．回転量は 16, 12, 8, 7 の 4 種類である．QuarterRound 構造では ARX 構造を 4 回繰り返す．そのため，QuarterRound 構造が 20 ラウンドであり，各ラウンドが QuarterRound 構造を 4 回繰り返し，QuarterRound 構造の中で ARX 構造が 4 回繰り返されているため，512 ビットのデータブロック 1 つに対して，ARX 構造が 320 回演算される．

Algorithm 2 ARX structure

```
1: a += b; d ^= a; d <<<= 16;
2: c += d; b ^= c; b <<<= 12;
3: a += b; d ^= a; d <<<= 8;
4: c += d; b ^= c; b <<<= 7;
```

3. 関連研究

マルウェアの暗号アルゴリズムに着目した解析は主に動的解析と静的解析の 2 パターンある．動的解析は，マルウェアなどを実際に実行し，挙動をリアルタイムで解析する手法である．サンドボックスなどで実行し，プログラムの操作や通信などを分析する．一方静的解析は，コードを実行せずに構造を調べる手法である．

3.1 DBI を用いたランサムウェアのテイント解析

与那嶺らは，QEMU 上で実装されたバイナリ解析プラットフォームである PANDA を用いたテイント解析を行っ

© 2026 Information Processing Society of Japan

た [3]．対象のデータにテイントタグを付与し，データの伝搬を追跡することでデータと関連する命令の抽出を試みた．その結果，平文データから鍵に関係する命令群の抽出や関数コールスタックの解析により，暗号処理に関する関数を部分的に特定できることを示した．

3.2 ランサムウェア「REvil」の静的・動的解析研究

小林は，ランサムウェア「REvil」について静的解析・動的解析の 2 面から調査を行った [4]．この研究では静的解析と動的解析を組み合わせることで，ソフトウェアの基本的な構造から動作フローや暗号化の手順を詳しく示している．REvil は暗号化処理として楕円曲線暗号の Curve25519 とストリーム暗号の Salsa20 が用いられていて，その呼び出しプロセスや関数の処理内容を具体的に示した．

3.3 アセンブリ命令を用いた機械学習によるマルウェア検知手法

Liu らは，逆アセンブル結果から取得したオペコード列を用いて，命令列の並びから N-gram 特徴量を生成し，それらの特徴量を機械学習へ入力することで，マルウェアを検知する手法を提案した [5]．まとめる特徴量の数や機械学習の種類をいくつか組み合わせて実験を行い，正解率を指標に示している．実験の結果，Random Forest と 3-gram または 5-gram の組み合わせが最も高性能で正解率 97.77% と高い水準を示している．また，命令列にマルウェアの特徴が含まれていることを示した．

3.4 ランサムウェアの動的オペコード解析

Carlin らは，暗号化型ランサムウェアの実行時に取得したオペコード情報を用いた動的解析の有効性を検証した [6]．VirtualBox 上に構築した Windows 10 環境でランサムウェアを実行し，デバッガを用いて実行時のオペコードを取得した．そして，その情報を Random Forest に入力することで正常プログラムとの分類を行った．また，32,000 個のオペコードからなる短い実行トレースを用いた場合でも高い分類精度を示し，7zip による正常な暗号化・圧縮処理についてもランサムウェアと区別できることを示している．

3.5 本研究の位置づけ

既存研究では，ランサムウェアの動的解析・静的解析について多く研究されているが，本研究は解析対象に ChaCha20 を，解析手法として逆アセンブル結果および QEMU トレースを用いて静的・動的の両面から ARX 構造を検出する研究を行った．

4. 提案手法

本研究では，ChaCha20 アルゴリズムの ARX 構造に着目して，逆アセンブル結果および QEMU トレースから ARX 構造を検出する手法を提案する．

4.1 概要

逆アセンブル結果と QEMU トレースを組み合わせて ARX 構造を検出することを目的とする．逆アセンブル結果からプログラム全体の命令列とレジスタに着目し，ARX 構造を検出する．また，逆アセンブル結果で確認できた ARX 構造から機械語を抽出することで，QEMU トレースから ARX 構造を検出するための条件を作成する．QEMU トレースでは実行時の CPU 処理を追い，逆アセンブル結果から得られた検出条件の機械語に対応する行を ARX 構造の候補として検出する．また，ChaCha20 プログラムから採取した QEMU トレースと `ls` や `mkdir` などの通常コマンドから採取した QEMU トレースを比較することで，ChaCha20 特有の構造であることを確認する．

4.2 検出アルゴリズム

検出プログラムでは ChaCha20 の ARX 構造に着目し，`ror` とその回転量（ARM 環境は右回転のため，32 ビットから各左回転量を引いた 16, 20, 24, 25 がそれぞれ対応）を抽出し，その `ror` ごとに命令を遡り `eor`，`add` を見つける．この際 `add` を中心に，ARX 構造の `add` ではない別の処理の `add` が誤検出されてしまわないように，命令の実行結果が格納される宛先レジスタ `dst` と，演算対象となる値を保持する入力元レジスタ `src` の 2 つのレジスタに着目する．特に `ror` の `src` が `eor` の `dst` および `add` の `dst` と一致していることを確認する．下記のアルゴリズムは python 言語をもとに簡略化した検出アルゴリズムの一部である．回転命令が見つかった後に，その前の命令を探索し，`ror_src` と合致する `dst` をもつ `eor` を ARX 構造の `eor` として検出する．同様に `ror_src` と合致する `dst` を持つ `add` を ARX 構造の `add` として検出する．

Algorithm 3 Detection Algorithm

```
for i ← ror_idx − 1 to max(−1, ror_idx − 10) do
    if instructions[i] is not an EOR instruction then
        continue
    end if
    op_info ← extract_operand(instructions[i])
    if op_info = None then
        continue
    end if
    if op_info[dst] = ror_src then
        eor_inst ← instructions[i]
        eor_idx ← i
        eor_info ← op_info
        break
    end if
end for
```

© 2026 Information Processing Society of Japan

5. 検証

5.1 検証環境

検証環境は以下の表 1 と表 2 に示す環境を用いて行った．表 1 に示す PC1 が，逆アセンブル結果から ARX 構造を検出する際に使用した環境である．また，表 2 に示す PC2 が，QEMU にて ChaCha20 プログラムのコンパイルと逆アセンブルを行い，QEMU トレースを採取した際に使用した環境である．

表 1 PC1 の環境

項目	環境・バージョン
OS	Windows 11 Home 24H2
WSL	2.6.3.0
Python	3.12.3
arm-linux-gnueabihf-gcc	13.3.0
arm-linux-gnueabihf-objdump	2.42

表 2 PC2 の環境

使用ツール	使用環境・バージョン
OS	Ubuntu 24.04.3 LTS
QEMU	5.0.0
QEMU 内 gcc	8.3.0
QEMU 内 objdump	2.31.1

また，QEMU トレースの採取には，表 3 に示す環境を用いた．

表 3 QEMU の環境

QEMU 構成内容	各環境・バージョン
QEMU	5.0.0
アーキテクチャ	ARM
マシン	versatilepb
CPU	ARM1176
メモリ	256 MB
カーネル	Linux 4.19.50

5.2 ChaCha20 プログラムの取得

この提案手法では，ChaCha20 アルゴリズムで暗号化するプログラムが必要である．今回は GitHub から 1 件，ChaCha20 アルゴリズムを使用した暗号化プログラムを入手した [7]．対象のレポジトリから ChaCha20 アルゴリズムが記された ChaCha20.h と sample-cipher.c を用いた．

5.3 逆アセンブル結果の取得および検出プログラムの実装

入手したプログラムに対し，コンパイル・逆アセンブリを以下の方法で行った．ARM 環境を想定し `arm-linux-gnueabihf-gcc` でクロスコンパイル，`arm-linux-gnueabihf-objdump` で逆アセンブリを行った．また，コンパイルオプションについて今回は，最適化なしの`-O0` で実行した．

次に逆アセンブル結果に対する検出プログラムを実装した．初めに逆アセンブル結果を確認すると，図 1 のように `add`, `eors`, `mov.w （ror）` が確認できた．そのため，逆アセンブル結果の機械語から命令名を取り出し，`add`, `eor`, `ror` かどうかを判定した．次に，mov.w 命令のオペランドから `ror` の回転量を抽出し，`ror` および回転量を検出コードの基盤とした．`ror` がある地点から命令を遡っていき `eor`, `add` を検出し，n 命令以内にすべてそろえば ARX 構造の候補と検出されるコードである．しかし ARX 構造以外の `add` 演算子であっても ARX 構造とみなしてしまうという問題点が生じた．そこで，それぞれの演算対象のレジスタに着目し，同一のレジスタに対して演算を行っている `ror`, `eor`, `add` のみを ARX 構造の候補として検出するように改善した．

5.4 QEMU トレースの取得および検出プログラムの実装

CPU エミュレータである QEMU を用いて実行トレースを採取した．QEMU にて ChaCha20 プログラムをコンパイルし，実行ファイルを作成した．また，逆アセンブル結果を取得した．そして作成された実行ファイルの実行 1 回につき QEMU トレースを 1 つ採取した．

次に QEMU トレースに対する検出プログラムを実装した．QEMU トレースは命令名が文字列ではなく数値で記されているため，機械語が 16 進数で格納されている `op1` などの特徴量に注目して `add`, `eor`, `ror` に相当する命令および回転対象のレジスタと合致するレジスタをもつ行を抽出しながら検出プログラムの作成を行った．

6. 結果

6.1 逆アセンブル結果の解析

採取した逆アセンブル結果を目視で確認すると，`add`, `eor`, `ror` が図 1 のように発見できた．各行は，左から命令アドレス，機械語，ニーモニックおよびオペランドを表す．図 1 の 1 行目の命令アドレスが 6d4 である `add` 命令が ARX 構造の A に対応する加算命令である．また，3 行目の命令アドレスが 6e8 である `eors` 命令が ARX 構造の X に対応する排他的論理和である．そして命令アドレスが 6f6 である 5 行目が ARX 構造の R に対応する回転命令である．ここでは，`mov.w r2, r2, ror #16` と表されていることがわかる．なお，この図 1 では ARX 構造に関係しないと考えられる中間の命令を省略しているが，`adds`, `ldr`, `str` などが含まれていた．

また，逆アセンブル結果に対して，作成した検出プログラムを実行すると，表 4 の結果を得られた．`ror` 命令に着目しその回転量ごとの検出数も出力したが，各 80 件ずつ検出できていることがわかる．また，出力された ARX 構

© 2026 Information Processing Society of Japan

```
6d4: 441a          add r2, r3
 ... 9 instructions omitted
6e8: 404a          eors r2, r1
 ... 6 instructions omitted
6f6: ea4f 4232     mov.w     r2, r2, ror #16
```

図 1 逆アセンブル結果における目視での ARX 構造の検出例

造候補の検出例のうち先頭 10 件を確認すると，add, eor, ror 間で対象レジスタが一致していて，周辺に存在する他の add や eor 命令ではなく，図 1 で確認した実際の ARX 構造と対応していた．さらに，全体の検出件数もソースコードから想定される ARX 構造の出現回数と一致していることがわかる．これらより，今回の逆アセンブル結果から ARX 構造を適切に検出できている可能性が高いと考えられる．

表 4 逆アセンブル結果内の ARX 構造の検出結果

検出項目	検出数
命令数	9,180
ARX 構造数	320
ror #16	80
ror #20	80
ror #24	80
ror #25	80

6.2 QEMU トレース内の ARX 構造検出

QEMU で取得した逆アセンブル結果と QEMU トレースを，対応付けて確認すると，ARX 構造に対応する機械語の候補が挙げられた．QEMU で取得した逆アセンブル結果からは図 2 のように ARX 構造の候補が確認できた．出力形式は図 1 と同様であり，各行は左から命令アドレス，機械語，ニーモニックおよびオペランドを表す．

```
105c4: e0822003  add r2, r2, r3
... 9 instructions omitted
105ec: e0222001  eor r2, r2, r1
... 6 instructions omitted
10608: e1a02862  ror r2, r2, #16
```

図 2 QEMU で取得した逆アセンブル結果における ARX 構造の検出例

回転命令に着目すると，4 つの回転量ごとに同じ機械語が割り振られていた．回転量 16 に対して e1a02862，20 に対して e1a02a62，24 に対して e1a02c62，25 に対して e1a02ce2 が割り振られていて，それぞれ 80 件ずつ検出できた．そして，これらの回転命令を含む ARX 構造に対応する add, eor の機械語を辿っていくと，eor の機械語である e0222001 が 321 件確認できた．add の機械語は 2 種類あり，e0812002 が 281 件，e0822003 が 41 件確認できた．

QEMU トレースから回転命令の機械語を中心に，これらの機械語が一定範囲内に出現する箇所を ARX 構造の候補として検出した．今回はこの範囲を ror から eor までを 40 命令，eor から add までを 80 命令として設定した．ARX 構造検出プログラムを実行すると，表 5 が得られた．

表 5 QEMU トレース内の ARX 構造の検出結果

検出項目	検出数
命令数	1,638,867
ror 検出数	55,571
ARX 構造数	55,571
ror #16	13,929
ror #20	14,099
ror #24	13,804
ror #25	13,739

表 5 から見て取れるように，回転量ごとの ror 命令検出数はそれぞれ異なり，最も検出数の多い回転量 20 と最も検出数の少ない回転量 25 の差は 360 件となった．また，検出できた ror 命令数と同数の ARX 構造の候補が検出できている．ARX 構造の検出候補のうちの先頭 10 個を確認すると，各命令間の距離が add から eor までは 10 命令，eor から ror までは 7 命令となっていた．そのため，初めに設定した探索範囲は広く，誤検出の可能性が増えると考え，add から eor までを 15 命令，eor から ror までを 10 命令に設定した．すると，結果は ARX 構造の候補数が 52,951 件に減少した．このことから，ARX 構造候補の先頭 10 個は距離がそれぞれ 10, 7 命令であったが，すべての ARX 構造の距離が一定ではなかった．そのため，表 6 のように，距離を変えた時の検出数の変化について調査した．距離を add から eor までを 40，eor から ror までを 20 に設定し実行すると，55,425 件検出された．次に add から eor の間を 60，eor から ror の間を 30 に変更すると，55,571 件検出できた．このように今回の QEMU トレースに対しては，ARX 構造の最大距離が add から eor までが 60，eor から ror までが 30 が最大となったが，距離が一定ではないことを考え，80, 40 で設定した．

表 6 各 QEMU トレースからの ARX 構造の検出件数

add-eor の距離	eor-ror の距離	ARX 構造検出数
15	10	52,951
40	20	55,425
60	30	55,571
80	40	55,571

また，各 QEMU トレースからの検出数は表 7 のような結果となった．検出数は約 54,800 件前後であるが，00-2-1 だけその他より約 20,000 件ほど多く検出された．

© 2026 Information Processing Society of Japan

表 7　各 QEMU トレースからの ARX 構造の検出件数

QEMU トレース	ARX 構造検出数
O0-1-1	55,571
O0-2-1	72,855
O0-3-1	54,645

6.3 通常コマンドの QEMU トレースの検証結果

ChaCha20 で暗号化するプログラムの QEMU トレースから add, eor, ror が順番通りに高頻度で出現したことを確認した．次に，ARX 構造を用いない ls や mkdir などの通常コマンドに対して検出プログラムの挙動を検証した．QEMU トレースで cat, echo, ls -a, ls, pwd, cp, mkdir, mv, rm, touch の 10 種のコマンドから各 2 回ずつ採取した，計 20 個の QEMU トレースに対して検証を行った．検出プログラムの検証結果はすべて ARX 構造の検出数が 0 件となり，現在登録している命令パターンに一致する ARX 構造は検出されなかった．

7. 考察

7.1 逆アセンブル結果からの検出結果

逆アセンブル結果からの検出結果としては，1 ブロックで実行される ARX 構造の理論値である 320 件と同数の ARX 構造の候補が検出された．本手法では，命令名とレジスタの関係に基づき，一定範囲内で add, eor, ror が順番通りに出現する箇所を ARX 構造の候補として検出した．検出された候補のうち先頭 10 件について逆アセンブル結果の該当箇所を照合すると，実際の ARX 構造を構成する命令列と対応していることが確認できた．また，全体の検出件数も想定される件数と一致していることから，本実験で用いた逆アセンブル結果に対しては，提案手法により ARX 構造に対応する命令列を ARX 構造の候補として検出できている可能性が高いと考えられる．

7.2 QEMU トレースからの検出結果

QEMU トレースからの検出結果としては，回転量 16, 20, 24, 25 の ror 命令，eor 命令，add 命令のそれぞれの機械語を逆アセンブル結果から取得し，QEMU トレースの機械語列から一定範囲以内で add, eor, ror が出現する箇所を ARX 構造の候補として検出した．ただし，本手法は設定した機械語および命令間の位置関係に基づいて候補を検出するため，検出された候補が必ずしも実際の ARX 構造に対応するとは限らず，同一条件を満たす ARX 構造以外の命令列を検出する可能性がある．そこで，ls や mkdir などの通常コマンドから取得した QEMU トレースに対しても同様の検出を行った．その結果，これらの通常コマンドでは，本手法の検出条件に設定した機械語パターンを満たす命令列は検出されなかった．以上より，今回使用した通常コマンドと ARX 構造を含むプログラムとの間では，本手法による検出結果に差が確認された．このことから，add, eor, ror の出現順序や位置関係，回転量およびレジスタに基づく今回の検出条件が，ARX 構造または ARX 構造を含む処理に特徴的な命令列をとらえている可能性があると考えられる．

8. 結論

本研究では，GitHub から入手した 1 つの ChaCha20 暗号化プログラムに対して，ChaCha20 アルゴリズムの核となる ARX 構造を検出することを目的とした．結果としては逆アセンブル結果および QEMU トレースから一定範囲で演算対象レジスタが依存関係にある add, eor, ror が順番に出現する ARX 構造の候補を検出することができた．今回の論文では，検出パターンに制限があり，特に QEMU トレースからの検出では特定の機械語との一致で判断しているため，アーキテクチャの違いやレジスタの変化などによって機械語が異なる場合に対応できない．そのため，今後は本提案手法の拡張を進めていく予定である．

謝辞

本研究の一部は，JSPS 科研費 26K14827 の支援により行った．

参考文献

[1] S. Alzahrani, Y. Xiao, and W. Sun, *"An Analysis of Conti Ransomware Leaked Source Codes,"* IEEE Access, Vol. 10, pp. 100178-100193, 2022.

[2] RFC Trans, "RFC 8439 - ChaCha20 and Poly1305 for IETF Protocols 日本語訳 ," https://tex2e.github.io/rfc-translater/html/rfc8439.html (Accessed 2026-07-21).

[3] 与那嶺俊, 門林雄基, *"DBI を用いたランサムウェアのファイルアクセス解析の一検討,"* 信学技報, Vol. 117, No. 127, ICSS2017-18, pp. 87-92, 2017.

[4] 小林麟太郎, *"ランサムウェア「REvil」の静的・動的解析研究,"* 2022 年度情報処理学会関西支部支部大会講演論文集, 2022.

[5] J. Liu, L. Dang and X. Liu, *"An Assembly Instruction-Based Machine Learning Approach for Detecting Cryptocurrency Malware,"* Proceedings of the 3rd International Conference on Sensors, Electronics and Computer Engineering (ICSECE), pp. 362-366, 2025.

[6] D. Carlin, P. O'Kane and S. Sezer, *"Dynamic Opcode Analysis of Ransomware,"* Proceedings of the International Conference on Cyber Security and Protection of Digital Services (Cyber Security), pp. 1-4, 2018.

[7] M. Izquierdo, "ChaCha20," https://github.com/marcizhu/ChaCha20/ (Accessed 2026-07-21).

© 2026 Information Processing Society of Japan